



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Processamento de Linguagens

Trabalho Prático

Hélder Tiago Peixoto da Cruz

A104174

Orlando Daniel Venda da Costa

A104255

João Pedro Rodrigues Veloso

A104434

1. Introdução

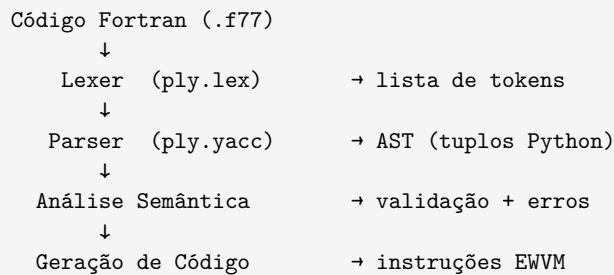
Este relatório descreve o desenvolvimento de um compilador para um subconjunto de operações da linguagem Fortran 77, no âmbito da unidade curricular de Processamento de Linguagens do ano letivo 2025/2026. O compilador traduz código Fortran para instruções da máquina virtual EWVM disponibilizada pelos docentes.

O compilador foi implementado em Python, recorrendo às ferramentas **ply.lex** e **ply.yacc**. para as fases de análise léxica e sintática, respetivamente.

O documento está organizado da seguinte forma: a secção 2 descreve a arquitetura geral do compilador; as secções 3 a 6 detalham cada fase do pipeline; a secção 7 apresenta os testes realizados; a secção 8 descreve as limitações conhecidas e a secção 9 conclui com uma reflexão sobre o trabalho desenvolvido.

2. Arquitetura do Compilador

O compilador segue um pipeline clássico de quatro fases sequenciais:



2.1. Representação Interna — AST

A árvore sintática abstrata (AST) é representada através de **tuplos Python**, onde o primeiro elemento identifica o tipo de nó. Por exemplo:

```
('assign', 'FAT', ('binop', '*', ('id', 'FAT'), ('id', 'I')))\n('do', 10, 'I', ('number', 1), ('id', 'N'), [...])\n('if', ('relop', '<=.', ...), [...], None)
```

2.2. Formato do Código Fonte

O enunciado deixa ao critério do grupo a escolha entre o formato de colunas fixas do Fortran 77 original e o formato livre. Optou-se pelo **formato livre**, por simplificar a análise léxica e sintática e por ser suficiente para os exemplos trabalhados. Assim, o compilador não interpreta posições fixas de colunas nem regras de continuação próprias do formato clássico.

3. Análise Lexical

O analisador léxico foi implementado com `ply.lex` e é responsável por converter o código fonte Fortran numa sequência de tokens.

3.1. Tokens Suportados

Os tokens reconhecidos pelo lexer dividem-se nas seguintes categorias:

- **Palavras-chave:** `PROGRAM`, `INTEGER`, `REAL`, `LOGICAL`, `IF`, `THEN`, `ELSE`, `ENDIF`, `DO`, `CONTINUE`, `GOTO`, `FUNCTION`, `SUBROUTINE`, `CALL`, `RETURN`, `READ`, `PRINT`, `END`
- **Identificadores:** sequências começadas por letra, podendo conter letras, algarismos e `_`, sendo insensíveis a maiúsculas e minúsculas
- **Literais numéricos:** inteiros e reais em formato simples
- **Literais textuais:** strings delimitadas por plicas
- **Literais lógicos:** `.TRUE.` e `.FALSE.`
- **Operadores relacionais:** `.EQ.`, `.NE.`, `.LT.`, `.LE.`, `.GT.`, `.GE.`
- **Operadores lógicos:** `.AND.`, `.OR.`, `.NOT.`
- **Operadores aritméticos:** `+`, `-`, `*`, `/`
- **Símbolos:** `=`, `(`, `)`, `,`

3.2. Decisões Relevantes

Os identificadores são convertidos para maiúsculas, pois assim o compilador passa a tratar nomes como `soma`, `SOMA` e `Soma` como o mesmo identificador, aproximando-se um pouco do comportamento tradicional de Fortran.

Decidimos também que os comentários iniciados por `!` são ignorados pelo lexer, assim como espaços e tabulações. Esta decisão simplificou as fases seguintes do desenvolvimento do projeto, uma vez que o parser recebe apenas tokens relevantes para a estrutura do programa.

Outra das decisões que tomamos foi tratar os valores `.TRUE.` e `.FALSE.` como literais lógicos, enquanto `.AND.`, `.OR.` e `.NOT.` são tratados como operadores lógicos. Achamos esta distinção importante para a análise semântica pois permite diferenciar valores booleanos de operações sobre valores booleanos.

4. Análise Sintática

O analisador sintático foi implementado com `ply.yacc`, construindo a **AST** a partir da sequência de tokens produzida pelo lexer.

4.1. Gramática

As principais produções da gramática são apresentadas abaixo:

```
program      → PROGRAM ID statements END opt_subprogram_list

statements   → statements statement | statement

statement    → declaration | assignment | print_stmt | read_stmt
              | if_stmt | do_stmt | goto_stmt | call_stmt
              | return_stmt | continue_stmt
              | NUMBER labeled_statement_core

declaration  → INTEGER decl_list | REAL decl_list | LOGICAL decl_list
decl_item    → ID | ID LPAREN NUMBER RPAREN

assignment   → ID ASSIGN expression
              | array_access ASSIGN expression

print_stmt   → PRINT TIMES COMMA print_list
read_stmt    → READ TIMES COMMA read_list

if_stmt      → IF LPAREN condition RPAREN THEN statements ENDIF
              | IF LPAREN condition RPAREN THEN statements ELSE statements ENDIF

do_stmt      → DO NUMBER ID ASSIGN expression COMMA expression
              do_body_statements labeled_continue

goto_stmt    → GOTO NUMBER
call_stmt    → CALL ID LPAREN opt_arg_list RPAREN

function_decl → type_spec FUNCTION ID LPAREN opt_param_list RPAREN function_body END
subroutine_decl → SUBROUTINE ID LPAREN opt_param_list RPAREN function_body END

expression   → expression PLUS expression
              | expression MINUS expression
              | expression TIMES expression
              | expression DIVIDE expression
              | MINUS expression
              | NUMBER
              | ID
              | array_access
              | function_call
              | MOD LPAREN expression COMMA expression RPAREN
              | DOT_TRUE
              | DOT_FALSE

condition    → expression relop expression
              | condition DOT_AND condition
              | condition DOT_OR condition
              | DOT_NOT condition
              | expression
```

Algumas construções, como `SUBROUTINE` e `CALL`, são reconhecidas ao nível sintático e semântico, mas não são traduzidas para EWVM.

4.2. Decisões de Representação

A AST foi mantida simples, usando tuplos Python. Por exemplo, uma atribuição é representada por um nó da forma: ('assign', nome_variavel, expressao) e uma expressão aritmética por: ('binop', operador, esquerda, direita)

Os acessos a arrays e as chamadas de função usam uma forma sintática parecida, baseada em `ID(...)`. Assim, expressões como `NUMS(I)` e `DOBRQ(X)` são inicialmente próximas na AST, sendo depois distinguidas pela análise semântica e pela geração de código. Esta decisão simplifica a gramática, embora nos tenha obrigado nas fases seguintes a consultar a tabela de símbolos para perceber se o identificador corresponde a um array ou a uma função.

Os statements com labels, como `10 CONTINUE`, são também representados explicitamente na AST. Isto permitiu validar posteriormente se os labels usados em `GOTO` ou `DO` existem e se não existem duplicados.

4.3. Precedência de Operadores

A precedência dos operadores foi definida da seguinte forma, do menor para o maior:

Nível	Operadores
1 (menor)	COND_EXPR
2	.OR.
3	.AND.
4	.NOT.
5	.EQ. .NE. .LT. .LE. .GT. .GE.
6	+ -
7	* /
8 (maior)	UMINUS

O operador `UMINUS` representa o menos unário, usado em expressões como `-N` ou `-3`. Este operador tem precedência superior à multiplicação e divisão, para garantir que a expressão é interpretada corretamente antes de ser usada noutras operações.

4.4. Exemplo de AST

Para o programa do fatorial, o statement `DO 10 I = 1, N` gera a seguinte AST:

```
('do', 10, 'I',
  ('number', 1),
  ('id', 'N'),
  [
    ('assign', 'FAT', ('binop', '*', ('id', 'FAT'), ('id', 'I')))
  ]
)
```

5. Análise Semântica

A análise semântica percorre a AST e verifica a coerência do programa sem a modificar. Os erros encontrados são acumulados numa lista e reportados no final. Esta fase não altera a estrutura da AST, apenas constrói tabelas de símbolos, infere tipos e acumula erros encontrados ao longo da análise.

5.1. Verificações Realizadas

- **Declaração de variáveis:** todas as variáveis usadas devem estar previamente declaradas
- **Declarações repetidas:** uma variável não pode ser declarada mais do que uma vez no mesmo contexto
- **Compatibilidade de tipos:** atribuições entre valores do mesmo tipo são aceites; adicionalmente, `REAL ← INTEGER` é permitido, mas o contrário não
- **Condições:** a condição de um `IF` deve produzir um valor lógico
- **Arrays:** o tamanho declarado deve ser positivo; o índice deve ser `INTEGER`; usar um array sem índice numa expressão é considerado erro
- **Labels:** os labels devem ser inteiros, não podem estar duplicados e os targets de `GOTO` devem existir no programa
- **Ciclos DO:** a variável de controlo deve estar declarada e ser escalar numérica
- **Operador MOD:** apenas é aceite com argumentos `INTEGER`
- **Menos unário:** apenas pode ser aplicado a expressões `INTEGER` ou `REAL`
- **Subprogramas:** o número e os tipos dos argumentos são verificados nas chamadas
- **RETURN:** só é válido dentro de funções ou subrotinas

5.2. Tabela de Símbolos

A tabela de símbolos é representada por um dicionário Python que associa cada identificador à respetiva informação semântica. Para variáveis, é guardado o tipo e a espécie do símbolo, por exemplo `scalar` para variáveis simples ou `array` para vetores unidimensionais.

5.3. Inferência de Tipos

A análise semântica infere o tipo das expressões para validar operações e orientar a geração de código. Literais inteiros produzem `INTEGER`, literais reais produzem `REAL` e `.TRUE.` / `.FALSE.` produzem `LOGICAL`.

Em expressões aritméticas, `INTEGER` com `INTEGER` mantém o tipo `INTEGER`, enquanto qualquer operação com `REAL` produz `REAL`. Operações aritméticas com valores `LOGICAL` são rejeitadas.

6. Geração de Código

O backend gera diretamente instruções EWVM a partir da AST.

6.1. Layout de Memória Global

As variáveis globais ocupam slots sequenciais na zona global da EWVM, inicializados antes da instrução `START`. Cada variável escalar ocupa um slot e é acedida através de `PUSHG` e `STOREG`. Arrays unidimensionais globais são reservados como blocos contíguos. Para arrays de `INTEGER` ou `LOGICAL`, o backend usa `PUSHN`; para arrays de `REAL`, as posições são inicializadas individualmente com `PUSHF 0.0`.

```
// Para: INTEGER N, I, FAT
PUSHI 0    // N → offset 0
PUSHI 0    // I → offset 1
PUSHI 0    // FAT → offset 2
START
```

6.2. Tradução de Construções

6.2.1. Atribuição

```
FAT = FAT * I
```

```
PUSHG 2    // carrega FAT
PUSHG 1    // carrega I
MUL
STOREG 2   // guarda em FAT
```

6.2.2. Ciclo DO

```
DO 10 I = 1, N
    FAT = FAT * I
10 CONTINUE
```

```
PUSHI 1
STOREG 1    // I = 1
LO:
PUSHG 1     // I
PUSHG 0     // N
INFEQ      // I <= N ?
JZ LBL10
    PUSHG 2
    PUSHG 1
    MUL
    STOREG 2
PUSHG 1
PUSHI 1
ADD
```

```
STOREG 1      // I++
JUMP L0
LBL10:
```

6.2.3. IF-THEN-ELSE

```
IF (N .GT. 0) THEN
    PRINT *, 'positivo'
ELSE
    PRINT *, 'nao positivo'
ENDIF
```

```
PUSHG 0
PUSHI 0
SUP
JZ else_label
    PUSHES "positivo"
    WRITES
    WRITELN
JUMP end_label
else_label:
    PUSHES "nao positivo"
    WRITES
    WRITELN
end_label:
```

6.2.4. Funções

As funções são emitidas depois do corpo principal do programa, após a instrução `STOP`. Desta forma, a execução normal termina antes das definições das funções, que apenas são executadas quando chamadas explicitamente.

Na chamada de uma função, os argumentos são primeiro avaliados e colocados na stack de operandos. Depois o endereço da função é colocado com `PUSHA` e a chamada é feita com `CALL`. No caso de funções com vários parâmetros, o backend limpa os argumentos adicionais que já não são necessários após a chamada.

```
PUSHG 0
PUSHG 1
PUSHA FUNCCONVRT
CALL
SWAP
POP 1
STOREG 2
```

Dentro da função, os parâmetros são acedidos relativamente ao frame pointer. O valor de retorno é guardado numa posição local associada ao nome da função e, antes do `RETURN`, é colocado na posição reservada para devolver o resultado ao caller.

```
FUNCCONVRT:
PUSHI 0
// corpo da função
```



```
PUSHL 0
STOREL -1
RETURN
```

6.3. I/O

A instrução `READ` é traduzida para `READ + ATOI` ou `ATOF` consoante o tipo da variável. `PRINT` é traduzido para `WRITEI`, `WRITEF` ou `WRITES` consoante o tipo, seguido de `Writeln`.

7. Testes

Foram testados os cinco programas de exemplo fornecidos no enunciado, bem como testes unitários adicionais para construções específicas. A bateria de testes contém 52 testes e foi executada com sucesso.

7.1. Exemplos do Enunciado

Exemplo	Construções testadas	Resultado
Olá Mundo	<code>PRINT</code> , strings	Sucesso
Fatorial	<code>DO</code> , atribuição, <code>READ</code> , <code>PRINT</code>	Sucesso
Primo	<code>LOGICAL</code> , <code>.AND.</code> , <code>IF-ELSE</code> , <code>MOD</code>	Sucesso
Soma de array	arrays, <code>DO</code> , <code>READ</code> , <code>PRINT</code>	Sucesso
Conversor	função com múltiplos parâmetros	Sucesso

7.2. Como Correr os Testes

```
.venv/bin/python -m unittest discover -v tests
```

Além dos testes unitários, os cinco exemplos principais foram compilados com sucesso para código da EWVM disponibilizada pela cadeira.

7.3. Como Correr o Compilador

```
python compiler.py examples/primo.f -o examples/primo.vm
```

Com o ambiente virtual usado no desenvolvimento, o comando equivalente é:

```
.venv/bin/python compiler.py examples/primo.f -o examples/primo.vm
```

Nota - Se a opção `-o` não for indicada, o código EWVM é escrito no stdout.

8. Limitações

O compilador implementa um subconjunto de Fortran 77. As principais limitações conhecidas são:

- **SUBROUTINE não suportado no backend EWVM:** a construção é reconhecida ao nível sintático e semântico, mas não é traduzida para código EWVM
- **Formato fixo não suportado:** o compilador aceita apenas formato livre, sem tratamento das colunas fixas do standard original de Fortran 77
- **MOD apenas com INTEGER:** a função intrínseca MOD não suporta argumentos REAL
- **Arrays multidimensionais não suportados:** apenas arrays unidimensionais globais e estáticos são suportados
- **Arrays locais em funções não suportados:** o backend EWVM não gera código para arrays declarados dentro de funções
- **Sem verificação de limites em arrays:** o compilador não gera instruções de verificação de índices durante a execução
- **I/O simplificado:** apenas são suportadas instruções na forma `READ *, ...` e `PRINT *, ...`
- **Ciclos DO simplificados:** apenas é suportado incremento implícito de 1, sem passo explícito
- **Literais reais simples:** não é suportada notação exponencial, como `1.0E-3`
- **Sem otimização de código:** o código EWVM gerado é correto, mas não otimizado

9. Conclusão

O compilador desenvolvido é capaz de processar programas Fortran 77 em formato livre, cobrindo as principais construções definidas no enunciado: declaração de tipos e variáveis, expressões aritméticas, lógicas e relacionais, controlo de fluxo (`IF-THEN-ELSE`, `DO`, `GOTO`) e operações de I/O com `READ` e `PRINT`.

Entre as funcionalidades implementadas destacam-se o suporte para variáveis `INTEGER`, `REAL` e `LOGICAL`, arrays unidimensionais, expressões com conversão implícita de `INTEGER` para `REAL`, labels, funções com múltiplos parâmetros e a função intrínseca `MOD` para valores inteiros.

As principais dificuldades encontradas estiveram relacionadas com a adaptação da geração de código à EWVM, em particular na tradução de funções, labels, operações lógicas e arrays. Também foi necessário garantir que o compilador rejeita explicitamente construções ainda não suportadas, como `SUBROUTINE` e `MOD` com argumentos `REAL`, em vez de gerar código inválido para a máquina virtual.

Como trabalho futuro, seria possível estender o compilador para suportar subrotinas, arrays multidimensionais, formato fixo de Fortran 77, verificação de limites em arrays e otimizações adicionais no código EWVM gerado.